

Tables

When to use this

Tables help present structured data (logs, metrics, inventories) with aligned columns and optional theming. They are perfect for dashboards and admin tools where readability matters.

Key types

- `TableView` – Defines headers, column widths, and rows for rendering tabular data.
- `TableRenderOptions` – Controls padding, alignment, and border characters.
- `RenderOptions` – Supplies fallback colours for headers and borders.

Minimal example

```
var table = new TableView();
table.AddHeaders("Service", "Status", "Duration");
table.AddRow(new[] { "Auth", "OK", "120 ms" });
table.AddRow(new[] { "Payments", "Degraded", "450 ms" });
table.AddRow(new[] { "Search", "Failed", "-" });

table.TableRenderOptions.HeaderScheme = new ColorScheme(ConsoleColor.White, ConsoleColor.DarkBlue);
table.TableRenderOptions.WhitespaceCharacter = '.';
table.TableRenderOptions.TableWidthPercentage = 0.9m;

table.Render();
```

Advanced tips

- **Window width** – Pass explicit console dimensions to avoid wrapping on narrow terminals. This also simplifies unit tests. Adjust `TableRenderOptions.TableWidthPercentage` or `MaximumTableWidth` to cap the rendered width regardless of buffer size, and recompute widths when you detect `Console.WindowResized` events.
- **Whitespace character & borders** – Configure `TableRenderOptions.WhitespaceCharacter` to customise the filler between columns (e.g., dots or dashes). Swap out border characters through `TableRenderOptions.Lines` to emulate markdown tables or CSV-style separators, and disable outer borders entirely for log-style output.
- **Styling per row** – Toggle between `ContentScheme1`, `ContentScheme2`, and `SelectionScheme` to highlight warnings or active rows. You can set these schemes per render pass to reflect live status changes, and use `SelectionScheme` when the table accompanies a `SelectionPrompt` so the highlighted row aligns with the user's current choice.

- **Export & composition** – Use `table.Render()` to a `StringWriter` if you need to export formatted tables into logs or files. Embed table output inside views or routines to render summaries before prompting the user for next steps, or stash the rendered string in clipboard/notification integrations.
- **Sorting & paging** – Preprocess your data before adding rows; many consumers combine `TableView` with LINQ to support custom sorting, filtering, or paging scenarios within dynamic views. Persist the source data separately from the rendered rows so you can rebuild the table when filters change without losing formatting rules.
- **Conditional columns** – Build helper methods that add optional columns only when data is present (for example, `table.AddOptionalColumn("Latency", values)`). This keeps the table compact on narrow terminals while still providing detail in rich environments.